



Fast File Search Tool

Subtitle: High-Performance Information Retrieval for Desktop Systems

Project Name

Fast File Search Tool

Subject

Information Retrieval & Text Mining

Team

Ebad Junaid(BSCS22046), Muhammad Shahzaib
Hafeez(BSCS22104), Afaq Buzdaar(BSCS22080),
Muhammad Yasir Zahid(BSCS22102)

Tagline

"Bringing Search Engine speeds to the Desktop File
System."

The Problem Statement

The Issue:



Slow Native Search

Standard OS tools (Windows Explorer, macOS Spotlight) can be sluggish or resource-heavy when scanning large directories.



Blocking Operations

Most simple scripts use **Linear Search ($O(N)$)**, scanning files one by one. This causes the "Application Not Responding" freeze on large datasets.



Lack of Control

Existing tools are often closed-source or platform-specific (e.g., "Everything" is Windows-only).

The Goal:

Build a cross-platform tool that indexes 100,000+ files instantly and allows for real-time, non-blocking search.

Project Objectives

1

Performance

- Index 100,000+ files in under 7 seconds.
- Return search results in **< 100 milliseconds** (Real-time).

2

Technique

Apply **Information Retrieval (IR)** concepts (Inverted Indexing) to file systems, moving away from brute-force scanning.

3

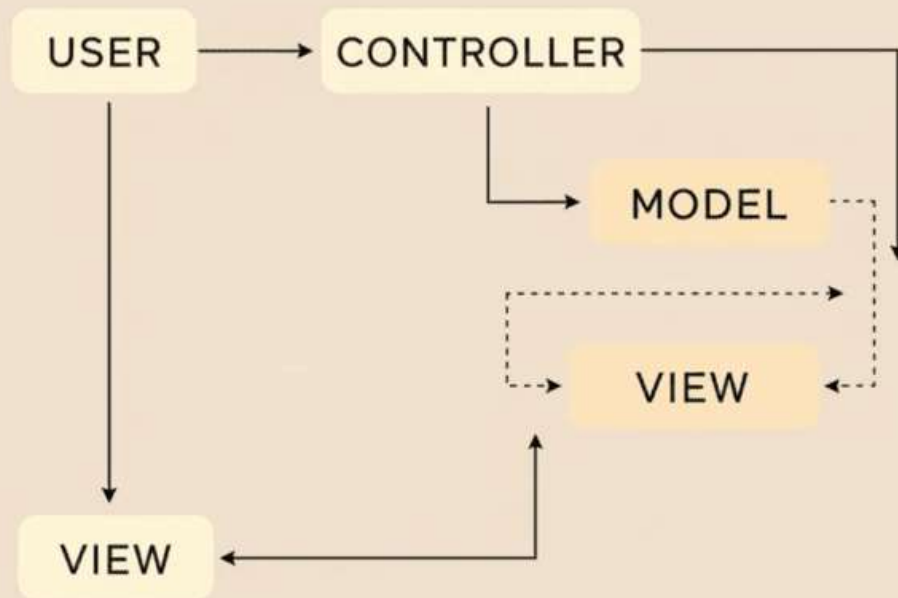
User Experience (UX)

- "Search-as-you-type" functionality.
- Hybrid View: Native File Browser + Instant Search List.
- Zero UI freezing during heavy indexing.

System Architecture (MVC Pattern)

Shutterstock

MODEL - VIEW - CONTROLLER PATTERN



Model (The Brain)

- Stores the **Inverted Index** and File Metadata.
- Handles the logic of `os.walk` and data structuring.

View (The Face)

- Built with **PyQt6** for native rendering on macOS/Windows.
- Displays the Folder Tree (Browser Mode) and Result List (Search Mode).

Controller (The Glue)

- Connects user input to the backend logic.
- **Multithreading:** Uses `QThread` to run the Indexer on a background thread, ensuring the Main UI Thread remains responsive.

Core IR Concept — The Inverted Index

Definition:

A data structure that maps "terms" to the "documents" that contain them.

Why we use it:



Implementation:

We use a Python Dictionary (HashMap) where:

- **Key:** A specific Trigram (e.g., dat).
- **Value:** A "Postings List" of Document IDs (e.g., {1, 5, 20}).

Core IR Concept — N-Gram Tokenization

The Challenge:

Users rarely type full filenames. They type substrings (e.g., "port" for "Report").

The Solution:

We use **Trigrams (3-Grams)**.

Process:

- **Input:** report.pdf
- **Normalization:** Convert to lowercase $\rightarrow$ report.pdf
- **Tokenization:** Break into 3-char overlapping shingles:
 - rep, epo, por, ort, rt., t.p, .pd, pdf

Benefit:

This allows us to match partial queries instantly without scanning the whole string.

Core IR Concept — Boolean Retrieval Model

Logic:

Our search engine uses **Conjunctive Queries** (The **AND** operator).

Scenario:

User searches for data.

1. Step 1: Decompose query into trigrams $\rightarrow$ dat, ata.

2. Step 2: Retrieve Postings Lists:

- dat $\rightarrow$ {Doc 1, Doc 5, Doc 99}
- ata $\rightarrow$ {Doc 5, Doc 20}

3. Step 3: Perform Set Intersection:

- $\{1, 5, 99\} \cap \{5, 20\} = \{5\}$

Result:

Only Document 5 matches *both* parts of the query.

Core IR Concept — Ranking & Verification

False Positive Elimination:

1. Trigrams are probabilistic. banana and bandana share many trigrams.
2. **Verification Step:** After the index retrieval, we perform a quick exact substring check to ensure 100% precision.

Ranking Strategy:

1. **Exact Match:** Files matching the query exactly appear at the top.
2. **Length Normalization:** Shorter filenames appear higher (assuming less noise).
3. **Alphabetical:** Tie-breaker sorting.

The Workflow (Crawler to Search)

01

Crawling

- The IndexerWorker traverses the directory tree using `os.walk`.
- Collects metadata: Path, Size, Date, Extension.

02

Indexing

- Assigns a unique DocID to every file.
- Generates Trigrams and populates the Inverted Index in memory.

03

Searching

- User input triggers the `search_index` function.
- Retrieves candidate IDs via Set Intersection.
- UI updates the Result List.

Fast File Search Tool

Technical Stack

- **Language:** Python 3.11+
- **GUI Framework:** PyQt6
- Selected over Tkinter to solve macOS Dark Mode issues and support complex widgets like QTreeView.
- **Data Structures:**
 - defaultdict(set): For the Inverted Index (handling sparse data efficiently).
 - Counter: For calculating file extension frequencies.
- **OS Integration:**
 - subprocess: To handle "Reveal in Finder/Explorer" commands natively.



User Experience (UX) Features

Hybrid View Architecture:

- **Browser Mode:** When idle, the app acts as a native File Explorer with a full folder tree structure.
- **Search Mode:** As soon as typing begins, the view swaps to a flat, prioritized list of search results.

Smart Filtering:

The "File Type" dropdown isn't just alphabetical; it sorts extensions by **Frequency** (e.g., if you have mostly PDFs, .pdf appears at the top).

Context Menus:

Right-click support to "Copy Path" or "Reveal File".



Challenges & Solutions

Challenge 1: The "Frozen UI"

Issue: Indexing large folders blocked the main application loop.

Solution: Implemented **Multithreading**. The Indexer runs on a separate worker thread and communicates progress via Signals.

Challenge 2: macOS Dark Mode

Issue: Standard UI libraries rendered invisible text on dark backgrounds.

Solution: Migrated to PyQt6, which respects system themes and provides native rendering.

Challenge 3: "Blank Start" State

Issue: The app looked broken when launched without a folder selected.

Solution: Added a "Clean Start" state with clear instructions and a placeholder view.

Live Demo Checklist

(Speaker notes for the live demonstration)

01

Launch:

Show the clean startup screen.

03

Browser Mode:

Show navigating the folder tree like a normal Explorer.

05

Search:

Type a query (e.g., "rep") and show the instant switch to the Result List.

02

Select Folder:

Pick the large "Dataset" folder.

04

Indexing:

Point out the progress bar working in the background.

06

Context Menu:

Right-click a file and choose "Reveal in Finder".



Future Enhancements

- **Content Indexing:** Extend the crawler to read the *inside* of PDF/TXT files using libraries like PyPDF2.
- **Persistent Storage:** Save the Inverted Index to disk (JSON/Pickle) so the user doesn't have to re-index on every launch.
- **File Watching:** Use watchdog observers to update the index automatically when files are created or deleted.

Conclusion

We successfully built a **High-Performance, IR-Based** search tool.

We moved beyond simple scripting to a robust **MVC Architecture**.

We solved real-world usability problems (Freezing, Cross-Platform UI) while maintaining the theoretical rigor of **Boolean Retrieval** and **Inverted Indexing**.

Resources

- Github link [<https://github.com/EbadJunaid/Fast-file-search-tool>]
- Recording link [https://drive.google.com/drive/folders/1TwvQ0oXWFnkvZXPAPVbjp8jbGAwLgu?usp=drive_link]